

Student-Scale Persistent 3D World Modeling on MOVi-A

Memory-Conditioned ConvGRU with Uncertainty-Gated Writes
CS 599 G1 – Final Project Report

Jitarth Bhavesh Patel
Boston University
jitarth@bu.edu

Abstract

We study *student-scale* persistent 3D world modeling on the MOVi-A benchmark, focusing on whether explicit persistent memory improves next-view RGB prediction over a non-persistent baseline. We implement a three-stage ConvGRU pipeline: (i) a no-memory baseline that predicts future frames from context only, (ii) a memory-conditioned model that is augmented with a rendered persistent 3D memory built from context observations, and (iii) a memory-conditioned model with an uncertainty head that gates memory writes. Across our completed experiments, the memory-conditioned model reduces overall validation L1 error from 0.0216 to 0.0189 and improves PSNR from 25.31 dB to 26.42 dB versus a no-memory baseline at the same compute budget, while reducing dynamic-region L1 from 0.103 to 0.089. The uncertainty-gated variant is currently still training and is reported as in-progress. We additionally implemented a diffusion variant of the same pipeline; under our compute budget the diffusion branch did not converge to usable quality and is reported transparently as an attempted extension rather than a primary result.

1 Introduction

Long-horizon visual prediction in 3D scenes requires a model to keep track of what it has already seen, where the camera was, and how the world looked at earlier time steps. Purely frame-local predictors and even short-context recurrent predictors collapse under viewpoint change, occlusion, and accumulating uncertainty. Recent work on *persistent* 3D world models [1] addresses this by maintaining an explicit, geometrically grounded memory of the scene that the model can read from when predicting new views.

These models are typically trained at scales well beyond what is practical inside a one-semester graduate course. The goal of this project is therefore not to reproduce them at full scale, but to ask a narrower and more student-tractable question:

At a small scale, on MOVi-A, does adding an explicit persistent memory improve a recurrent next-view predictor over a matched no-memory baseline?

We answer this with a compact ConvGRU-based pipeline that we run entirely on a single workstation. Our contributions are:

1. A reproducible end-to-end pipeline that goes from raw MOVi clips to trained checkpoints and qualitative comparison artifacts using only local scripts.
2. A controlled comparison between a no-memory ConvGRU baseline and a memory-conditioned ConvGRU under matched training budgets, dataset splits, and hyperparameters.

3. An uncertainty-aware extension that estimates a per-pixel confidence and uses it to gate memory writes; we describe its intended behavior, training setup, and the qualitative artifacts already produced (final quantitative results in-progress at submission time).
4. An honest, compute-aware account of an attempted diffusion variant of the same task: our scripts, the configuration we attempted, and the reasons it did not converge to usable quality in our setting.

2 Related Work

Persistent 3D world models. Recent work formulates world modeling not just as next-frame prediction but as maintaining a persistent 3D representation that observations are written into and future views are queried from [1]. This is the conceptual starting point for our pipeline.

Recurrent video prediction. Convolutional recurrent architectures, such as ConvLSTM [3] and ConvGRU [4], are a standard backbone for spatiotemporal prediction at modest scale. We use a ConvGRU because it offers a favorable accuracy/compute tradeoff for student-scale training.

Diffusion video models. Conditional video diffusion is the state-of-the-art family for high-fidelity generative video prediction [5]. We treat it as an attempted extension to our pipeline; in our compute regime it did not converge within the time available, which we discuss in Section 4.5.

Uncertainty-aware prediction. Heteroscedastic uncertainty heads [6] provide a way to predict per-pixel confidence alongside the prediction itself. We use this idea both as an auxiliary loss and as a gating signal for what the model writes into its persistent memory.

Dataset. We use the MOVi-A subset of the Kubric [2] synthetic benchmark, which provides multi-view RGB-D videos with ground-truth camera poses and intrinsics — ideal for studying memory-conditioned prediction at small scale.

3 Task and Notation

Let a clip provide RGB frames $\{I_t\}_{t=1}^T$, depth maps $\{D_t\}_{t=1}^T$, camera poses $\{P_t\}_{t=1}^T$, and intrinsics K . We split each clip into windows of length $T_c + T_p$ where the first T_c frames form the *context* and the next T_p frames form the *target* that the model must predict.

The model receives the context RGB $\{I_t\}_{t=1}^{T_c}$, the context poses $\{P_t\}_{t=1}^{T_c}$, and the target poses $\{P_t\}_{t=T_c+1}^{T_c+T_p}$, and outputs predicted RGB $\{\hat{I}_t\}_{t=T_c+1}^{T_c+T_p}$. Memory-conditioned variants also receive a rendered memory tensor (Section 4.3).

In all our final runs we use $T_c = T_p = 4$ and an image resolution of 64×64 .

4 Method

4.1 Data Pipeline

We export MOVi-A clips into compressed `.npz` shards containing RGB video, depth, camera poses, camera intrinsics, and metadata. A manifest file lists all clip shards and is the only source of truth that downstream training scripts read. From these clips we extract overlapping context/target

windows; the number of windows per clip is capped separately for the train and validation splits to control runtime. We use a fixed random seed and a fixed `val_ratio = 0.2` to split clips into train and validation sets, so every model variant sees identical data.

4.2 No-Memory ConvGRU Baseline

Each context frame is encoded by a small convolutional encoder. The resulting feature sequence is passed through a ConvGRU temporal cell [4] that produces a hidden state. The hidden state is then decoded, conditioned on each target pose, into the predicted target RGB sequence. The training loss is a weighted combination of per-pixel L1 reconstruction and a dynamic-region L1 term that up-weights pixels where the target frame moves relative to the last context frame.

This baseline isolates the value of *recurrence alone* on this task and serves as the reference for the memory-augmented models.

4.3 Memory-Conditioned ConvGRU

The memory-conditioned model adds an explicit persistent 3D scene memory. From the context RGB-D and poses we construct a voxelized memory grid of resolution $48 \times 40 \times 48$. For each target pose, this grid is differentially rendered into target-view-aligned RGB, depth, and a coverage mask. The model receives the context features (as in the baseline) and additionally a rendered memory conditioning tensor for each target frame.

The training objective for this model adds, on top of the baseline RGB and dynamic-region losses, (i) an auxiliary depth L1 loss and (ii) a memory-covered L1 loss that focuses error on regions for which the memory render returned a non-empty pixel. This pushes the model to actually *use* the memory render rather than ignore it.

The memory model is warm-started from the converged no-memory ConvGRU: weights with matching shapes (encoder, ConvGRU cell, RGB decoder, etc.) are copied over so that the memory model only has to learn the new memory pathway, not the entire predictor. This is a practical choice that strongly stabilizes training under our compute budget.

4.4 Uncertainty-Gated Memory Writes

The uncertainty variant adds a heteroscedastic head [6] that predicts a per-pixel log-variance. During training we add a Gaussian negative log-likelihood term whose weight, write-confidence threshold, and confidence gamma are all exposed as hyperparameters. At rollout time, the predicted confidence can be used to *gate* which pixels are written into the persistent memory: low-confidence pixels are down-weighted or skipped, which is intended to prevent the memory from being contaminated by uncertain estimates.

The uncertainty model is warm-started from the converged memory-conditioned ConvGRU. We follow the same data and optimization settings as the memory model, plus the additional uncertainty hyperparameters. At the time of report submission, the uncertainty training run is still in progress; we therefore report only its setup and qualitative artifacts, and explicitly mark its quantitative cell in Table 1 as in progress.

4.5 Diffusion Variant (Attempted)

We additionally implemented no-memory and memory-conditioned diffusion variants of the same task using a small conditional video diffusion model. Configurations matched the ConvGRU pipeline as closely as possible (same data manifest, image size, T_c , T_p). Diffusion training, however,

requires substantially more optimizer steps (target was 100 epochs at $\sim 20,000$ optimizer steps each) and the per-step cost is dominated by repeated denoising sampling at evaluation time. Within our available compute budget, diffusion runs did not reach a stable, usable quality regime; we therefore do *not* report diffusion numbers in our main quantitative table. The scripts and exact configurations remain in the repository so that the diffusion experiments are fully reproducible if more compute becomes available.

5 Experimental Setup

Dataset and split. We use 200 MOVi-A clips at 128×128 source resolution, split with a fixed seed into 160 train clips and 40 validation clips. From these we extract up to 16 train and 8 validation context/target windows per clip.

Architecture. Each ConvGRU model uses `hidden_channels` = 128. Memory-conditioned variants use a memory grid of (48, 40, 48), memory stride 1, and splat radius 1.

Training schedule. The no-memory and memory models are trained for 50 epochs each with batch size 16. The uncertainty variant is trained for up to 40 epochs warm-starting from the memory checkpoint. We use Adam with the script’s default learning rate (10^{-3}) and report *best validation* checkpoints chosen by the “`model_l1`” metric.

Loss configuration.

- Dynamic-region weight: 3.0 (motion threshold 0.03).
- Depth loss weight (memory variants): 0.1.
- Memory-covered loss weight (memory variants): 1.0.
- Memory render loss weight (memory variants): 0.0.
- Uncertainty NLL weight (uncertainty variant): 0.25, with write-confidence threshold 0.99 and confidence gamma 4.0.

Hardware and runtime. All reported numbers come from single-GPU training (we used both a workstation GPU and an L40S-class GPU). The dominant runtime cost is per-sample memory conditioning (voxel write + render), which significantly increases the cost of each training and validation pass for the memory-conditioned models relative to the no-memory baseline.

6 Results

6.1 Quantitative Comparison

Table 1 summarizes the validation results from the final `metrics.json` of each completed run. The last-frame baseline is the same simple “copy last context frame” predictor that the training script already evaluates against, and is reported once for reference; it is identical for every model variant because it does not depend on a learned model.

Model	Epochs	Val L1 ↓	Dyn L1 ↓	PSNR (dB) ↑	Beats baseline?
Last-frame baseline	—	0.0200	0.1280	24.55	—
No-memory ConvGRU	50	0.0216	0.1029	25.31	dyn only
Memory ConvGRU	50	0.0189	0.0890	26.42	yes
Memory + uncertainty	in progress	in progress	in progress	in progress	—

Table 1: Validation results on the MOVi-A subset200 split. Lower is better for L1 metrics, higher is better for PSNR. Numbers come from the `final_val` block of each run’s `metrics.json`. The uncertainty row is intentionally left as “in progress” because that training run had not finished at the time of submission.

Reading the table.

- The *no-memory ConvGRU* matches or improves on the last-frame baseline on dynamic regions (lower dynamic L1) and on PSNR, but does not improve global L1. This is consistent with a model that learns reasonable motion but introduces small errors in static regions where the last-frame baseline is, by construction, near-perfect.
- The *memory ConvGRU* improves on the no-memory model on every reported axis: -12.5% on validation L1, -13.5% on dynamic L1, $+1.1$ dB PSNR. It also beats the last-frame baseline on global L1, dynamic L1, and the memory-covered L1 metric simultaneously.
- The *uncertainty* row is intentionally pending; we defer its quantitative discussion to the camera-ready version of this report once the training run completes.

6.2 Qualitative Comparison

Figures 1 and 2 show best-validation comparison panels for the no-memory and memory ConvGRU models on the same validation example. Each panel stacks the context strip, the ground-truth target strip, the model prediction strip, and the last-frame baseline strip.

Visually, the memory model preserves more high-frequency object detail and tracks moving objects more cleanly, which is consistent with its lower dynamic-region L1 in Table 1.

6.3 Uncertainty-Gated Variant: In-Progress Diagnostics

While final quantitative numbers for the uncertainty variant are not yet available, the warm-started run already produces the diagnostic strips that justify the architectural choice. Figure 3 shows the predicted confidence strip and the resulting binary write-mask strip from a representative validation window: the model concentrates uncertainty on object boundaries and disoccluded regions, which is exactly where one would want to suppress memory writes.

6.4 Diffusion Outcome

Diffusion variants of the same pipeline were launched but did not reach a usable quality regime within our compute budget; we therefore omit them from quantitative comparison and from main figures. We retain their training scripts in the repository so that (i) the methodology is documented and (ii) anyone with more compute can reproduce the attempted setup directly.

7 Discussion

Memory helps, even at this scale. The most important observation is that explicit persistent memory provides measurable gains over a matched no-memory ConvGRU at the same training budget. The improvement is largest on dynamic-region L1, which is consistent with the intuition that memory helps most where the last-frame baseline fails — in regions that are moving or recently disoccluded.

Warm-starting matters. Training the memory model from random weights was much less stable than training it from the no-memory checkpoint. Warm-starting effectively decomposes the problem into “learn a video predictor” and then “learn to use the memory pathway,” which is a useful design pattern for student-scale training.

Uncertainty pattern is the right shape. Even mid-training, the predicted confidence concentrates exactly where one would expect (motion boundaries, depth discontinuities). This is encouraging evidence that uncertainty-gated writing is a sensible mechanism, even if final quantitative ablations require longer training.

Diffusion is compute-bound, not idea-bound. The diffusion variant ran on the same data and same downstream metric but simply did not have time to converge. We do not interpret this as evidence against diffusion; we interpret it as a clear scope limit of this project.

8 Limitations

- **Compute budget.** All training fits inside a student-scale workstation/GPU budget, which limited hyperparameter sweeps and prevented diffusion convergence.
- **Resolution.** Training at 64×64 is enough to rank model variants but is not high enough to produce photorealistic outputs; this constrains qualitative claims.
- **Subset size.** We use a 200-clip MOVi-A subset, not the full benchmark. Numbers should be read as a ranking rather than as state-of-the-art absolute metrics.
- **In-progress uncertainty results.** Final quantitative results for the uncertainty variant are not yet included; only setup and qualitative diagnostics are reported.

9 Conclusion

We presented a student-scale, end-to-end pipeline for persistent 3D world modeling on MOVi-A. The memory-conditioned ConvGRU clearly outperforms a matched no-memory baseline on validation L1, dynamic L1, and PSNR, while also producing visibly sharper qualitative predictions. An uncertainty-gated extension is in progress and already produces interpretable confidence patterns. A diffusion variant of the same pipeline was implemented but did not converge within our compute budget and is reported transparently as future work.

Reproducibility

- Data preparation and environment setup are documented in `README.md` of the project repository.

- Training is launched with `scripts/train_convgru_all.sh` (the full ConvGRU pipeline) or with the convenience wrapper `scripts/train_full_pipeline.sh`.
- Diffusion attempts use `scripts/train_diffusion_all.sh`.
- Per-run `metrics.json`, comparison images, and checkpoints are saved under `outputs/` with stable directory names so that figures and tables in this report can be regenerated without code changes.

References

- [1] Anonymous authors. “Learning 3D Persistent Embodied World Models.” Reference paper, 2025.
- [2] Greff, Klaus, et al. “Kubric: a scalable dataset generator.” *CVPR*, 2022.
- [3] Shi, Xingjian, et al. “Convolutional LSTM Network: A Machine Learning Approach for Precipitation Nowcasting.” *NeurIPS*, 2015.
- [4] Ballas, Nicolas, et al. “Delving Deeper into Convolutional Networks for Learning Video Representations.” *ICLR*, 2016.
- [5] Ho, Jonathan, et al. “Video Diffusion Models.” *NeurIPS*, 2022.
- [6] Kendall, Alex, and Yarin Gal. “What Uncertainties Do We Need in Bayesian Deep Learning for Computer Vision?” *NeurIPS*, 2017.



Figure 1: No-memory ConvGRU: best-validation comparison panel. Top to bottom: context, target, model prediction, last-frame baseline.



Figure 2: Memory-conditioned ConvGRU: best-validation comparison panel for the same task. Compared to Figure 1, predictions are sharper and motion in dynamic regions is reproduced more faithfully.



Figure 3: Uncertainty variant, in-progress diagnostics. Left: predicted per-pixel confidence (brighter = more confident). Right: derived write mask (white = allowed to update memory). The pattern matches expectations: low confidence at motion and depth boundaries.